
System Zarządzania Infrastrukturą VPN

Dokumentacja Techniczna Projektu

Zespół projektowy:

Wiktor Banek Bartłomiej Bieda
Daniel Kubiela Oskar Jasiak

Spis treści

1	Wprowadzenie i analiza wymagań	3
1.1	Cel projektu	3
1.2	Zakres funkcjonalny	3
1.3	Skład zespołu i podział odpowiedzialności	3
2	Projekt logiczny – diagram ERD	4
2.1	Opis encji i relacji	4
2.2	Relacje między tabelami	4
3	Projekt fizyczny – schemat bazy danych	5
3.1	Tabela uzytkownicy	5
3.2	Tabela klucze_dostepu	5
3.3	Tabela serwery_vpn	5
3.4	Tabela tunele_vpn	5
3.5	Tabela logi_polaczen	6
3.6	Tabela pomiary_parametrow	6
4	Normalizacja	6
4.1	Pierwsza postać normalna (1NF)	6
4.2	Druga postać normalna (2NF)	6
4.3	Trzecia postać normalna (3NF)	6
5	Implementacja – kod SQL	7
5.1	Tworzenie tabel	7
6	Wyzwalacze (Triggers)	8
6.1	Wyzwalacz sprawdzający ważność klucza	8
6.2	Wyzwalacz blokujący jednoczesne sesje	9
6.3	Wyzwalacz automatycznej rotacji kluczy	10
7	Widoki	10
7.1	Widok v_aktywne_polaczenia	10
7.2	Widok v_audyt_bezpieczenstwa	11
7.3	Widok v_wydajnosc_infrastruktury	11
8	Transakcje i poziomy izolacji	11
8.1	Zastosowanie transakcji	11
8.1.1	Scenariusz 1 – Rejestracja nowego użytkownika z kluczem	12
8.1.2	Scenariusz 2 – Zamknięcie sesji	12
8.2	Poziomy izolacji	12
9	Bezpieczeństwo danych – role i uprawnienia	13
9.1	Struktura ról w systemie PostgreSQL	13
9.2	Przyznane uprawnienia	13
9.3	Zasada najmniejszych uprawnień	14

10 Zaawansowane zapytania SQL	14
10.1 Zagnieżdżone podzapytanie – użytkownicy bez aktywnej sesji	14
10.2 Statystyki sesji per użytkownik	15
10.3 Najwydajniejsze tunele VPN	15
10.4 Wygasające klucze – raport dla administratora	15
11 Dane testowe i studium przypadku	16
11.1 Dane testowe	16
11.2 Studium przypadku 1: Rola vpn_readonly (Audytor)	17
11.3 Studium przypadku 2: Rola vpn_operator (Serwis L1)	17
12 Podsumowanie	17

1 Wprowadzenie i analiza wymagań

1.1 Cel projektu

Celem projektu jest zaprojektowanie i implementacja relacyjnej bazy danych dla systemu zarządzania infrastrukturą VPN (*Virtual Private Network*). System umożliwia obsługę użytkowników, zarządzanie kluczami dostępu, monitorowanie aktywnych połączeń tunelowych oraz zbieranie danych o wydajności sieci.

1.2 Zakres funkcjonalny

System realizuje następujące funkcje:

1. **Zarządzanie użytkownikami** – rejestracja, przechowywanie danych logowania i adresu e-mail.
2. **Zarządzanie kluczami dostępu** – generowanie hasha klucza, ustawianie daty wygaśnięcia, aktywacja/deaktywacja.
3. **Infrastruktura serwerowa** – ewidencja serwerów VPN wraz z adresem IP, lokalizacją i dostawcą hostingu.
4. **Tunele VPN** – obsługa protokołów OpenVPN, WireGuard i IPsec na dedykowanych portach.
5. **Logowanie połączeń** – rejestrowanie czasu rozpoczęcia i zakończenia sesji z przypisanym kluczem i tunelem.
6. **Pomiary wydajności** – zbieranie opóźnienia (ms) i przepustowości (Mbps) dla każdej sesji.
7. **Bezpieczeństwo** – wyzwalacz blokujący połączenia z nieaktywnymi lub wygasłymi kluczami; podział ról w bazie danych.

1.3 Skład zespołu i podział odpowiedzialności

Projekt został zrealizowany w zespole czteroosobowym. Zgodnie z wytycznymi, każdy członek zespołu był liderem wyznaczonego obszaru prac, odpowiadając za pełny cykl życia przypisanych komponentów:

- **Wiktor Banek (wbanek)** – Koordynator projektu, odpowiedzialny za analizę wymagań systemowych, przygotowanie schematu koncepcyjnego, normalizację bazy do 3NF oraz fizyczną implementację struktury tabel, kluczy i sekwencji.
- **Daniel Kubiela (dkubiela)** – Analityk danych, odpowiedzialny za projektowanie logicznych relacji, implementację zaawansowanych widoków systemowych oraz optymalizację zapytań agregujących z wieloma połączeniami.
- **Oskar Jasiak (ojasiak)** – Programista bazy danych, odpowiedzialny za automatyzację logiki biznesowej, implementację funkcji proceduralnych w języku PL/pgSQL oraz wdrożenie wyzwalaczy systemowych.

- **Bartłomiej Bieda (bbieda)** – Administrator systemu, odpowiedzialny za warstwę bezpieczeństwa (zarządzanie rolami i uprawnieniami), kontrolę transakcji oraz konfigurację poziomów izolacji chroniących przed anomaliami.

2 Projekt logiczny – diagram ERD

2.1 Opis encji i relacji

Schemat bazy danych składa się z **sześciu tabel** powiązanych relacjami jeden-do-wielu:

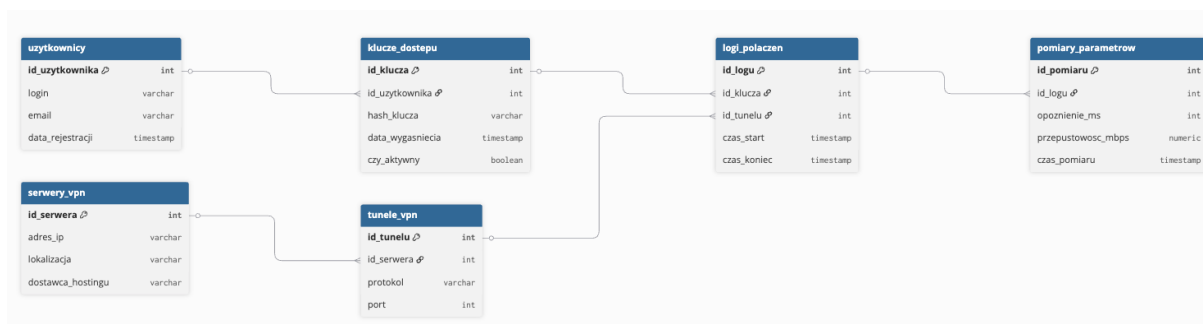
Tabela 1: Zestawienie encji

Encja	Klucz główny	Opis
uzytkownicy	id_uzytkownika	Użytkownicy systemu
klucze_dostepu	id_klucza	Klucze dostępu VPN
serwery_vpn	id_serwera	Serwery infrastruktury
tunele_vpn	id_tunelu	Tunele na serwerach
logi_polaczen	id_logu	Sesje połączeń
pomiary_parametrow	id_pomiaru	Pomiary jakości połączenia

2.2 Relacje między tabelami

Tabela 2: Klucze obce i relacje

Tabela podrzędna	Klucz obcy	Tabela nadrzędna	ON DELETE
klucze_dostepu	id_uzytkownika	uzytkownicy	CASCADE
tunele_vpn	id_serwera	serwery_vpn	CASCADE
logi_polaczen	id_klucza	klucze_dostepu	—
logi_polaczen	id_tunelu	tunele_vpn	—
pomiary_parametrow	id_logu	logi_polaczen	CASCADE



Rysunek 1: Diagram związków encji (ERD) dla systemu zarządzania VPN

3 Projekt fizyczny – schemat bazy danych

3.1 Tabela uzytkownicy

Tabela 3: Struktura tabeli uzytkownicy

Kolumna	Typ	Ograniczenia	Opis
id_uzytkownika	INTEGER	PK, AUTO	Identyfikator
login	VARCHAR(50)	NOT NULL, UNIQUE	Login
email	VARCHAR(100)	NOT NULL, UNIQUE	Adres e-mail
data_rejestracji	TIMESTAMP	DEFAULT NOW()	Data rejestracji

3.2 Tabela klucze_dostepu

Tabela 4: Struktura tabeli klucze_dostepu

Kolumna	Typ	Ograniczenia	Opis
id_klucza	INTEGER	PK, AUTO	Identyfikator
id_uzytkownika	INTEGER	FK, ON DEL CASCADE	Właściciel
hash_klucza	VARCHAR(255)	NOT NULL, UNIQUE	Skrót klucza
data_wygasniecia	TIMESTAMP	NOT NULL	Data wygaśnięcia
czy_aktywny	BOOLEAN	DEFAULT TRUE	Status klucza

3.3 Tabela serwery_vpn

Tabela 5: Struktura tabeli serwery_vpn

Kolumna	Typ	Ograniczenia	Opis
id_serwera	INTEGER	PK, AUTO	Identyfikator
adres_ip	VARCHAR(15)	NOT NULL, UNIQUE	Adres IP
lokalizacja	VARCHAR(100)	NOT NULL	Lokalizacja
dostawca_hostingu	VARCHAR(100)	–	Dostawca

3.4 Tabela tunele_vpn

Tabela 6: Struktura tabeli tunele_vpn

Kolumna	Typ	Ograniczenia	Opis
id_tunelu	INTEGER	PK, AUTO	Identyfikator
id_serwera	INTEGER	FK, ON DEL CASCADE	Serwer nadrzędny
protokol	VARCHAR(20)	NOT NULL, CHECK	OpenVPN/WireGuard/IPsec
port	INTEGER	NOT NULL	Port nasłuchu

3.5 Tabela logi_polaczen

Tabela 7: Struktura tabeli logi_polaczen

Kolumna	Typ	Ograniczenia	Opis
id_logu	INTEGER	PK, AUTO	Identyfikator
id_klucza	INTEGER	FK	Użyty klucz
id_tunelu	INTEGER	FK	Użyty tunel
czas_start	TIMESTAMP	DEFAULT NOW()	Początek sesji
czas_koniec	TIMESTAMP	NULL	Koniec sesji (NULL = aktywna)

3.6 Tabela pomiary_parametrow

Tabela 8: Struktura tabeli pomiary_parametrow

Kolumna	Typ	Ograniczenia	Opis
id_pomiaru	INTEGER	PK, AUTO	Identyfikator
id_logu	INTEGER	FK, ON DEL CASCADE	Sesja
opoznienie_ms	INTEGER	NOT NULL, ≥ 0	Opóźnienie [ms]
przepustowosc_mbps	NUMERIC(8,2)	NOT NULL, ≥ 0	Przepustowość [Mbps]
czas_pomiaru	TIMESTAMP	DEFAULT NOW()	Czas pomiaru

4 Normalizacja

4.1 Pierwsza postać normalna (1NF)

Wszystkie atrybuty we wszystkich tabelach przechowują wartości atomiczne (brak kolumn wielowartościowych, brak powtarzających się grup). Każda tabela posiada jednoznaczny klucz główny.

4.2 Druga postać normalna (2NF)

W żadnej tabeli nie występuje złożony klucz główny – każda tabela ma klucz syntetyczny (SERIAL), więc nie może wystąpić częściowa zależność funkcyjna.

4.3 Trzecia postać normalna (3NF)

Zbadano zależności przechodnie:

- klucze_dostepu: id_uzytkownika $\rightarrow$ dane użytkownika są w osobnej tabeli uzytkownicy – brak tranzytywności.
- tunele_vpn: dane serwera trzymane są w serwery_vpn – brak redundancji.
- pomiary_parametrow: id_logu $\rightarrow$ dane sesji w logi_polaczen – brak tranzytywności w tabeli pomiarów.

Żadna kolumna niebędąca kluczem nie zależy od innej kolumny niebędącej kluczem.

5 Implementacja – kod SQL

5.1 Tworzenie tabel

```
1 CREATE TABLE public.uzytkownicy (  
2     id_uzytkownika INTEGER NOT NULL,  
3     login          VARCHAR(50) NOT NULL,  
4     email          VARCHAR(100) NOT NULL,  
5     data_rejestracji TIMESTAMP WITHOUT TIME ZONE DEFAULT  
6         CURRENT_TIMESTAMP,  
7     CONSTRAINT uzytkownicy_pkey PRIMARY KEY (id_uzytkownika)  
8     ,  
9     CONSTRAINT uzytkownicy_login_key UNIQUE (login),  
10    CONSTRAINT uzytkownicy_email_key UNIQUE (email)  
11 );
```

Listing 1: Definicja tabeli uzytkownicy

```
1 CREATE TABLE public.klucze_dostepu (  
2     id_klucza      INTEGER NOT NULL,  
3     id_uzytkownika INTEGER,  
4     hash_klucza    VARCHAR(255) NOT NULL,  
5     data_wygasniecia TIMESTAMP WITHOUT TIME ZONE NOT NULL,  
6     czy_aktywny    BOOLEAN DEFAULT TRUE,  
7     CONSTRAINT klucze_dostepu_pkey PRIMARY KEY (id_klucza)  
8     ,  
9     CONSTRAINT klucze_dostepu_hash_klucza_key UNIQUE (hash_klucza),  
10    CONSTRAINT klucze_dostepu_id_uzytkownika_fkey  
11        FOREIGN KEY (id_uzytkownika)  
12        REFERENCES public.uzytkownicy(id_uzytkownika) ON DELETE  
13        CASCADE  
14 );
```

Listing 2: Definicja tabeli klucze_dostepu

```
1 CREATE TABLE public.tunele_vpn (  
2     id_tunelu      INTEGER NOT NULL,  
3     id_serwera     INTEGER,  
4     protokol       VARCHAR(20) NOT NULL,  
5     port           INTEGER NOT NULL,  
6     CONSTRAINT tunele_vpn_pkey PRIMARY KEY (id_tunelu),  
7     CONSTRAINT tunele_vpn_protokol_check  
8         CHECK (protokol IN ('OpenVPN', 'WireGuard', 'IPsec')),  
9     CONSTRAINT tunele_vpn_id_serwera_fkey  
10        FOREIGN KEY (id_serwera)  
11        REFERENCES public.serwery_vpn(id_serwera) ON DELETE CASCADE  
12 );
```

Listing 3: Definicja tabeli tunele_vpn

```
1 CREATE TABLE public.pomiary_parametrow (  
2     id_pomiaru      INTEGER NOT NULL,
```



```

3      id_logu          INTEGER,
4      opoznienie_ms    INTEGER NOT NULL,
5      przepustowosc_mbps NUMERIC(8,2) NOT NULL,
6      czas_pomiaru     TIMESTAMP WITHOUT TIME ZONE DEFAULT
          CURRENT_TIMESTAMP,
7      CONSTRAINT pomiary_parametrow_pkey PRIMARY KEY (id_pomiaru),
8      CONSTRAINT pomiary_parametrow_opoznienie_ms_check
          CHECK (opoznienie_ms >= 0),
9      CONSTRAINT pomiary_parametrow_przepustowosc_mbps_check
          CHECK (przepustowosc_mbps >= 0),
10     CONSTRAINT pomiary_parametrow_id_logu_fkey
          FOREIGN KEY (id_logu)
11     REFERENCES public.logi_polaczen(id_logu) ON DELETE CASCADE
12 );

```

Listing 4: Definicja tabeli pomiary_parametrow

6 Wyzwalacze (Triggers)

6.1 Wyzwalacz sprawdzający ważność klucza

Przed każdym wstawieniem rekordu do tabeli logi_polaczen wywoływana jest funkcja sprawdz_waznosc_klucza(). Sprawdza ona dwa warunki:

1. Czy klucz jest aktywny (czy_aktywny = TRUE).
2. Czy klucz nie wygasł (data_wygasniecia > CURRENT_TIMESTAMP).

W przypadku naruszenia dowolnego warunku operacja INSERT jest przerywana z komunikatem błędu.

```

1 CREATE FUNCTION public.sprawdz_waznosc_klucza() RETURNS trigger
2     LANGUAGE plpgsql AS
3 $$
4 DECLARE
5     wygasa    TIMESTAMP;
6     aktywny   BOOLEAN;
7 BEGIN
8     SELECT data_wygasniecia, czy_aktywny
9     INTO   wygasa, aktywny
10    FROM   klucze_dostepu
11    WHERE  id_klucza = NEW.id_klucza;
12
13    IF aktywny = FALSE THEN
14        RAISE EXCEPTION
15        'Odmowa dostępu: Klucz zostal dezaktywowany.';
16    ELSIF wygasa < CURRENT_TIMESTAMP THEN
17        RAISE EXCEPTION
18        'Odmowa dostępu: Klucz stracil waznosc.';
19    END IF;
20

```

```
21     RETURN NEW;  
22 END;  
23 $$;
```

Listing 5: Funkcja wyzwalacza sprawdz_waznosc_klucza()

```
1 CREATE TRIGGER trg_sprawdz_klucz  
2   BEFORE INSERT ON public.logi_polaczen  
3   FOR EACH ROW  
4   EXECUTE FUNCTION public.sprawdz_waznosc_klucza();
```

Listing 6: Rejestracja wyzwalacza weryfikującego ważność

6.2 Wyzwalacz blokujący jednoczesne sesje

Ze względów bezpieczeństwa oraz w celu zapobiegania współdzieleniu kont, system blokuje możliwość nawiązywania wielu jednoczesnych połączeń przy użyciu tego samego klucza. Wyzwalacz przed dodaniem nowego logu sprawdza, czy dany klucz posiada już aktywną sesję.

```
1 CREATE OR REPLACE FUNCTION public.blokuj_jednoczesne_sesje()  
2   RETURNS trigger  
3   LANGUAGE plpgsql AS  
4   $$  
5   BEGIN  
6     -- Sprawdzamy, czy ten klucz ma juz otwarta sesje  
7     IF EXISTS (  
8       SELECT 1 FROM public.logi_polaczen  
9       WHERE id_klucza = NEW.id_klucza  
10      AND czas_koniec IS NULL  
11    ) THEN  
12      RAISE EXCEPTION  
13        'Odmowa dostępu: Wykryto juz aktywna sesje VPN dla tego  
14        klucza.';  
15    END IF;  
16    RETURN NEW;  
17  END;  
18  $$;
```

Listing 7: Funkcja wyzwalacza blokuj_jednoczesne_sesje()

```
1 CREATE TRIGGER trg_blokuj_wielokrotne_sesje  
2   BEFORE INSERT ON public.logi_polaczen  
3   FOR EACH ROW  
4   EXECUTE FUNCTION public.blokuj_jednoczesne_sesje();
```

Listing 8: Rejestracja wyzwalacza blokady sesji

6.3 Wyzwalacz automatycznej rotacji kluczy

System automatycznie unieważnia stare klucze użytkownika w momencie przypisania mu nowego dostępu. Gwarantuje to, że w systemie aktywny jest zawsze maksymalnie jeden klucz per użytkownik.

```
1 CREATE OR REPLACE FUNCTION public.rotacja_kluczy_uzytkownika()  
  RETURNS trigger  
  LANGUAGE plpgsql AS  
2 $$  
3 BEGIN  
4   -- Dezaktywujemy wszystkie stare klucze tego uzytkownika  
5   UPDATE public.klucze_dostepu  
6   SET czy_aktywny = FALSE  
7   WHERE id_uzytkownika = NEW.id_uzytkownika  
8     AND id_klucza != NEW.id_klucza  
9     AND czy_aktywny = TRUE;  
10  
11   RETURN NEW;  
12 END;  
13 $$;
```

Listing 9: Funkcja wyzwalacza rotacja_kluczy_uzytkownika()

```
1 CREATE TRIGGER trg_rotacja_kluczy  
2   AFTER INSERT ON public.klucze_dostepu  
3   FOR EACH ROW  
4   EXECUTE FUNCTION public.rotacja_kluczy_uzytkownika();
```

Listing 10: Rejestracja wyzwalacza rotacji kluczy

7 Widoki

7.1 Widok v_aktywne_polaczenia

Zwraca listę aktualnie trwających sesji VPN (brak wartości czas_koniec). Łączy dane z czterech tabel: logi_polaczen, klucze_dostepu, uzytkownicy, tunele_vpn i serwery_vpn.

```
1 CREATE VIEW public.v_aktywne_polaczenia AS  
2 SELECT  
3   u.login,  
4   s.adres_ip,  
5   t.protokol,  
6   l.czas_start  
7 FROM logi_polaczen l  
8 JOIN klucze_dostepu k ON l.id_klucza = k.id_klucza  
9 JOIN uzytkownicy    u ON k.id_uzytkownika = u.id_uzytkownika  
10 JOIN tunele_vpn     t ON l.id_tunelu = t.id_tunelu  
11 JOIN serwery_vpn    s ON t.id_serwera = s.id_serwera  
12 WHERE l.czas_koniec IS NULL;
```

Listing 11: Widok aktywnych połączeń

7.2 Widok v_audyt_bezpieczenstwa

Widok audytowy – zwraca użytkowników, których klucz jest nieaktywny lub wygasa w ciągu 14 dni. Oblicza liczbę dni do wygaśnięcia.

```

1 CREATE VIEW public.v_audyt_bezpieczenstwa AS
2 SELECT
3     u.login,
4     u.email,
5     k.czy_aktywny           AS status_klucza,
6     k.data_wygasniecia,
7     DATE_PART('day',
8         k.data_wygasniecia::TIMESTAMPTZ - CURRENT_TIMESTAMP)
9         AS "dni_do_wygasniecia"
10 FROM uzytkownicy u
11 JOIN klucze_dostepu k ON u.id_uzytkownika = k.id_uzytkownika
12 WHERE k.czy_aktywny = FALSE
13        OR k.data_wygasniecia < (CURRENT_TIMESTAMP + INTERVAL '14 days')
14 ORDER BY k.data_wygasniecia;
```

Listing 12: Widok audytu bezpieczeństwa

7.3 Widok v_wydajnosc_infrastruktury

Agreguje dane wydajnościowe per lokalizacja i protokół: liczba sesji, średnie opóźnienie, szczytowa przepustowość.

```

1 CREATE VIEW public.v_wydajnosc_infrastruktury AS
2 SELECT
3     s.lokalizacja,
4     t.protokol,
5     COUNT(DISTINCT l.id_logu)           AS suma_sesji,
6     COALESCE(ROUND(AVG(p.opoznienie_ms),0),0)
7         AS srednie_opoznienie_ms
8     ,
9     COALESCE(MAX(p.przepustowosc_mbps),0) AS
10         szczytowa_przepustowosc_mbps
11 FROM serwery_vpn s
12 JOIN tunele_vpn t ON s.id_serwera = t.id_serwera
13 LEFT JOIN logi_polaczen l ON t.id_tunelu = l.id_tunelu
14 LEFT JOIN pomiary_parametrow p ON l.id_logu = p.id_logu
15 GROUP BY s.lokalizacja, t.protokol
16 ORDER BY srednie_opoznienie_ms DESC;
```

Listing 13: Widok wydajności infrastruktury

8 Transakcje i poziomy izolacji

8.1 Zastosowanie transakcji

System korzysta z transakcji PostgreSQL w scenariuszach wymagających atomowości. Poniższe przykłady ilustrują typowe scenariusze operacyjne.

8.1.1 Scenariusz 1 – Rejestracja nowego użytkownika z kluczem

```

1 BEGIN;
2
3 INSERT INTO uzytkownicy (login, email)
4 VALUES ('nowy_user', 'nowy@vpn.pl');
5
6 INSERT INTO klucze_dostepu
7     (id_uzytkownika, hash_klucza, data_wygasniecia)
8 VALUES (
9     currval('uzytkownicy_id_uzytkownika_seq'),
10    md5(random()::text),
11    CURRENT_TIMESTAMP + INTERVAL '1 year'
12 );
13
14 COMMIT;

```

Listing 14: Transakcja rejestracji użytkownika

8.1.2 Scenariusz 2 – Zamknięcie sesji

```

1 BEGIN;
2
3 UPDATE logi_polaczen
4     SET czas_koniec = CURRENT_TIMESTAMP
5 WHERE id_logu = 3
6     AND czas_koniec IS NULL;
7
8 INSERT INTO pomiary_parametrow
9     (id_logu, opoznienie_ms, przepustowosc_mbps)
10 VALUES (3, 22, 512.50);
11
12 COMMIT;

```

Listing 15: Transakcja zamknięcia sesji VPN

8.2 Poziomy izolacji

Tabela 9: Poziomy izolacji transakcji zastosowane w systemie

Poziom izolacji	Zastosowanie	Uzasadnienie
READ COMMITTED (domyślny)	Odczyty widoków, raporty	Wydajność, brak brudnych odczytów
REPEATABLE READ	Audyt bezpieczeństwa	Spójny obraz danych w trakcie raportu
SERIALIZABLE	Rejestracja klucza + połączenia	Pełna atomowość, brak anomalii

```

1 BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
2
3 INSERT INTO klucze_dostepu
4     (id_uzytkownika, hash_klucza, data_wygasniecia, czy_aktywny)
5 VALUES (1, 'nowy_bezpieczny_hash', NOW() + INTERVAL '6 months',
6         TRUE);
7
8 INSERT INTO logi_polaczen (id_klucza, id_tunelu)
9 VALUES (currval('klucze_dostepu_id_klucza_seq'), 2);
10 COMMIT;

```

Listing 16: Przykład transakcji z poziomem SERIALIZABLE

9 Bezpieczeństwo danych – role i uprawnienia

9.1 Struktura ról w systemie PostgreSQL

W celu odzwierciedlenia struktury zespołu oraz wdrożenia mechanizmów bezpieczeństwa, w systemie PostgreSQL zaimplementowano cztery konta użytkowników o zróżnicowanych poziomach uprawnień:

- **wbanek** – Główny właściciel bazy danych, posiadający pełne prawa DDL i DML.
- **dkubiela** – Rola dedykowana dla analityka, z uprawnieniami do odczytu i modyfikacji danych oraz pełnym dostępem do widoków.
- **ojasiak** – Konto deweloperskie z uprawnieniami proceduralnymi, umożliwiające tworzenie i modyfikację funkcji oraz wyzwalaczy (*TRIGGER*).
- **bbieda** – Rola administracyjna dedykowana do zarządzania uprawnieniami, kontroli transakcji oraz nadzoru nad sekwencjami.

9.2 Przyznane uprawnienia

W celu zapewnienia sprawnego środowiska deweloperskiego, właściciel bazy (**wbanek**) przyznał poszczególnym członkom zespołu uprawnienia administracyjne i operacyjne, ściśle dopasowane do powierzonych im zadań inżynierskich:

```

1 -- 1. Uprawnienia dla roli dkubiela
2 GRANT ALL ON TABLE public.uzytkownicy          TO dkubiela;
3 GRANT ALL ON TABLE public.klucze_dostepu        TO dkubiela;
4 GRANT ALL ON TABLE public.serwery_vpn           TO dkubiela;
5 GRANT ALL ON TABLE public.tunele_vpn            TO dkubiela;
6 GRANT ALL ON TABLE public.logi_polaczen          TO dkubiela;
7 GRANT ALL ON TABLE public.pomiary_parametrow     TO dkubiela;
8
9 -- Uprawnienia zwrotne do widokow (wlascicielem widokow jest
10 GRANT ALL ON TABLE public.v_aktywne_polaczenia   TO wbanek;

```

```
11 GRANT ALL ON TABLE public.v_audyt_bezpieczenstwa TO wbanek;  
12 GRANT ALL ON TABLE public.v_wydajnosc_infrastruktury TO wbanek;  
13  
14 -- 2. Uprawnienia dla roli ojasiak (Programista bazy - tworzenie  
    wyzwaczy)  
15 GRANT SELECT, INSERT, UPDATE, TRIGGER ON ALL TABLES IN SCHEMA  
    public TO ojasiak;  
16 GRANT ALL ON ALL FUNCTIONS IN SCHEMA public TO ojasiak;  
17  
18 -- 3. Uprawnienia dla roli bbieda (Administrator - zarządzanie  
    transakcjami)  
19 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public  
    TO bbieda;  
20 GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO bbieda;
```

Listing 17: Skrypt przyznawania uprawnień wewnątrz zespołu projektowego

9.3 Zasada najmniejszych uprawnień

W środowisku produkcyjnym zastosowano bardziej restrykcyjny podział uprawnień:

```
1 -- Rola tylko do odczytu (dla monitoringu)  
2 CREATE ROLE vpn_readonly;  
3 GRANT SELECT ON ALL TABLES IN SCHEMA public TO vpn_readonly;  
4 GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO vpn_readonly;  
5  
6 -- Rola operatora (insert/update, bez DDL)  
7 CREATE ROLE vpn_operator;  
8 GRANT SELECT, INSERT, UPDATE ON TABLE public.logi_polaczen  
9     TO vpn_operator;  
10 GRANT SELECT, INSERT, UPDATE ON TABLE public.pomiary_parametrow  
11     TO vpn_operator;  
12 GRANT SELECT ON TABLE public.klucze_dostepu TO vpn_operator;  
13 GRANT SELECT ON TABLE public.tunele_vpn TO vpn_operator;
```

Listing 18: Rekomendowana konfiguracja produkcyjna

10 Zaawansowane zapytania SQL

10.1 Zagnieżdżone podzapytanie – użytkownicy bez aktywnej sesji

```
1 SELECT u.login, u.email  
2 FROM uzytkownicy u  
3 WHERE u.id_uzytkownika NOT IN (  
4     SELECT k.id_uzytkownika  
5     FROM klucze_dostepu k  
6     JOIN logi_polaczen l ON k.id_klucza = l.id_klucza  
7     WHERE l.czas_koniec IS NULL  
8 );
```

Listing 19: Użytkownicy bez aktywnej sesji VPN

10.2 Statystyki sesji per użytkownik

```
1 SELECT
2     u.login,
3     COUNT(l.id_logu) AS liczba_sesji,
4     ROUND(SUM(
5         EXTRACT(EPOCH FROM (
6             COALESCE(l.czas_koniec, CURRENT_TIMESTAMP) - l.
              czas_start
7         )) / 60.0
8     )::numeric, 1) AS laczny_czas_min,
9     ROUND(AVG(p.przepustowosc_mbps), 2) AS
        srednia_przepustowosc_mbps
10 FROM uzytkownicy u
11 JOIN klucze_dostepu      k ON u.id_uzytkownika = k.id_uzytkownika
12 JOIN logi_polaczen      l ON k.id_klucza      = l.id_klucza
13 LEFT JOIN pomiary_parametrow p ON l.id_logu    = p.id_logu
14 GROUP BY u.login
15 ORDER BY laczny_czas_min DESC;
```

Listing 20: Łączny czas połączeń i średnia przepustowość na użytkownika

10.3 Najwydajniejsze tunele VPN

```
1 SELECT
2     s.lokalizacja,
3     t.protokol,
4     t.port,
5     COUNT(p.id_pomiaru) AS liczba_pomiarow,
6     ROUND(AVG(p.opoznienie_ms)) AS srednie_opoznienie_ms,
7     MAX(p.przepustowosc_mbps) AS max_przepustowosc_mbps
8 FROM tunele_vpn t
9 JOIN serwery_vpn      s ON t.id_serwera = s.id_serwera
10 LEFT JOIN logi_polaczen l ON t.id_tunelu = l.id_tunelu
11 LEFT JOIN pomiary_parametrow p ON l.id_logu = p.id_logu
12 GROUP BY s.lokalizacja, t.protokol, t.port
13 HAVING COUNT(p.id_pomiaru) > 0
14 ORDER BY srednie_opoznienie_ms ASC;
```

Listing 21: Ranking tuneli wg. średniego opóźnienia (rosnąco)

10.4 Wygasające klucze – raport dla administratora

```
1 SELECT
2     u.login,
```



```

3      u.email,
4      k.data_wygasniecia,
5      CEIL(EXTRACT(EPOCH FROM
6          (k.data_wygasniecia - CURRENT_TIMESTAMP)) / 86400)
7      AS dni_do_wygasniecia
8 FROM uzytkownicy      u
9 JOIN klucze_dostepu k ON u.id_uzytkownika = k.id_uzytkownika
10 WHERE k.czy_aktywny = TRUE
11      AND k.data_wygasniecia BETWEEN CURRENT_TIMESTAMP
12                                AND CURRENT_TIMESTAMP + INTERVAL '30
13                                days'
13 ORDER BY k.data_wygasniecia;

```

Listing 22: Klucze wygasające w ciągu 30 dni

11 Dane testowe i studium przypadku

11.1 Dane testowe

Baza zawiera predefiniowane dane testowe:

- **9 użytkowników** (m.in. admin_jan, devops_linux, stazysta_it).
- **9 kluczy dostępu** – różne stany: aktywny, nieaktywny, wygasły.
- **5 serwerów VPN** – Warszawa, Frankfurt, Kraków, Amsterdam, Nowy Jork; dostawcy: OVH, AWS, Comarch Cloud, DigitalOcean.
- **6 tuneli** – 3 protokoły: OpenVPN (port 1194), WireGuard (51820/51821), IPsec (500).
- **7 logów połączeń** – część aktywnych (bez czas_koniec), część zamkniętych.
- **8 pomiarów** – opóźnienia od 12 ms do 125 ms, przepustowość od 42 Mbps do 900 Mbps.

```

VPN_db=> SELECT
  (SELECT count(*) FROM uzytkownicy) as uzytkownicy_cnt,
  (SELECT count(*) FROM klucze_dostepu) as klucze_cnt,
  (SELECT count(*) FROM serwery_vpn) as serwery_cnt,
  (SELECT count(*) FROM tunele_vpn) as tunele_cnt,
  (SELECT count(*) FROM logi_polaczen) as logi_cnt,
  (SELECT count(*) FROM pomiary_parametrow) as pomiary_cnt;
uzytkownicy_cnt | klucze_cnt | serwery_cnt | tunele_cnt | logi_cnt | pomiary_cnt
-----
9 | 9 | 5 | 6 | 7 | 10
(1 row)

```

Rysunek 2: Weryfikacja danych testowych

11.2 Studium przypadku 1: Rola vpn_readonly (Audytor)

W architekturze bazy danych zastosowano model kontroli dostępu oparty na rolach (ang. *Role-Based Access Control*), realizujący Zasadę Najmniejszych Uprawnień.

Pierwsze studium weryfikuje konto z przeznaczeniem do audytu infrastruktury. Jak widać na dołączonym zrzucie ekranu (Rysunek 3), rola pomyślnie wykonuje zapytania selekcyjne (odczyt adresów IP serwerów), natomiast system restrykcyjnie blokuje wszelkie próby manipulacji stanem (odrzuć instrukcji UPDATE), chroniąc tym samym integralność bazy.

```
VPN_db=> \c VPN_db vpn_readonly
You are now connected to database "VPN_db" as user "vpn_readonly".
VPN_db=> SELECT adres_ip, lokalizacja FROM public.serwery_vpn LIMIT 2;
  adres_ip | lokalizacja
-----+-----
 192.168.1.100 | Warszawa, Polska
  10.0.5.50  | Frankfurt, Niemcy
(2 rows)

VPN_db=> UPDATE public.serwery_vpn SET lokalizacja = 'Zhakowana_Lokalizacja' WHERE id_serwera = 1;
ERROR: permission denied for table serwery_vpn
```

Rysunek 3: Weryfikacja uprawnień do odczytu i blokady modyfikacji dla audytora

11.3 Studium przypadku 2: Rola vpn_operator (Serwis L1)

Drugie studium przypadku udowadnia selektywne przypisanie uprawnień operacyjnych do konkretnych tabel.

Operator na pierwszej zmianie (L1) może zasilać tabele telemetryczne nowymi pomiarami sieciowymi (INSERT zakończony sukcesem), ale architektura sprzętowa odbiera mu możliwość usuwania jakichkolwiek logów historycznych z systemu. Ponadto potwierdzono prawidłową separację danych i izolację horyzontalną – rola techniczna nie posiada najmniejszych uprawnień odczytu do tabeli przechowujących dane personalne użytkowników, co stanowi kluczowy element zgodności z RODO (Rysunek 4).

```
VPN_db=> \c VPN_db vpn_operator
You are now connected to database "VPN_db" as user "vpn_operator".
VPN_db=> INSERT INTO pomiary_parametrow (id_logu, opoznienie_ms, przepustowosc_mbps)
VALUES (2, 25, 450.50);
INSERT 0 1
VPN_db=> DELETE FROM logi_polaczen WHERE id_logu = 2;
ERROR: permission denied for table logi_polaczen
VPN_db=> SELECT login, email FROM uzytkownicy;
ERROR: permission denied for table uzytkownicy
```

Rysunek 4: Weryfikacja uprawnień operacyjnych i restrykcyjnej separacji danych dla operatora

12 Podsumowanie

W ramach projektu zaprojektowano i zaimplementowano kompletny, wysoce zoptymalizowany system bazodanowy dedykowany do zarządzania oraz monitorowania rozproszonej

infrastruktury sieci VPN w środowisku PostgreSQL 13. Wszystkie założone cele inżynierskie oraz wymagania funkcjonalne zostały w pełni osiągnięte. Do najważniejszych efektów prac zespołu zaliczają się:

- **Pełna normalizacja (3NF)** – struktura wszystkich sześciu tabel została zoptymalizowana, eliminując redundancję danych oraz niepożądane zależności przechodnie, co zapewnia wysoką spójność strukturalną i wydajność zapytań.
- **Integralność referencyjna i więzy integralności** – powiązania relacyjne zostały zabezpieczone kluczami obcymi z precyzyjnie dobranymi regułami kaskadowymi (ON DELETE CASCADE). Poprawność wprowadzanych danych technicznych wymuszono za pomocą więzów CHECK.
- **Proceduralna automatyzacja bezpieczeństwa** – zaimplementowano zaawansowany wyzwalacz (*trigger*) powiązany z funkcją w języku PL/pgSQL, który w sposób automatyczny blokuje próby autoryzacji połączeń dla kluczy nieaktywnych bądź wygasłych, chroniąc infrastrukturę przed nieuprawnionym dostępem.
- **Zaawansowana analityka sieciowa** – utworzono trzy niezależne widoki relacyjne (*v_aktywne_polaczenia*, *v_audyt_bezpieczenstwa*, *v_wydajnosc_infrastruktury*), które dynamicznie agregują surowe dane i dostarczają gotowe raporty diagnostyczne w czasie rzeczywistym.
- **Bezpieczeństwo transakcyjne** – krytyczne operacje (takie jak rejestracja użytkownika czy zamykanie sesji sieciowych) ustrukturyzowano w bloki transakcyjne z jawnym przypisaniem poziomów izolacji (READ COMMITTED, REPEATABLE READ, SERIALIZABLE), całkowicie eliminując ryzyko wystąpienia anomalii współbieżności.
- **Strukturyzacja ról i uprawnień** – wdrożono kompletny model bezpieczeństwa odzwierciedlający strukturę i podział odpowiedzialności w zespole projektowym (*wbanek*, *dkubiela*, *ojasiak*, *bbieda*), zabezpieczając dostęp do poszczególnych warstw systemu za pomocą instrukcji GRANT.
- **Realistyczne środowisko testowe** – baza została zasilona spójnym zestawem danych testowych obejmującym: 9 użytkowników, 9 kluczy dostępu, 5 serwerów VPN, 6 tuneli, 7 logów połączeń sieciowych oraz 8 precyzyjnych pomiarów wydajnościowych.